Mixture of Tiny Experts for Resource-Constrained Vision-Language Models on Single Board Computer Clusters

Muhammad Ashir
Graduate School of Software and Information Science
Iwate Prefectural University
Iwate, Japan
s231x001@s.iwate-pu.ac.jp

Hirokazu Madokoro
Faculty of Software and Information Science
Iwate Prefectural University
Iwate, Japan
hirokazu_m@iwate-pu.ac.jp

Stephanie Nix
Faculty of Software and Information Science
Iwate Prefectural University
Iwate, Japan
nix_s@iwate-pu.ac.jp

Abstract

Deploying vision-language models (VLMs) on edge devices is challenging due to high computational and memory demands. In this paper, we propose **MoTE-VLM**, a Mixture of Tiny Experts architecture for efficient VLM inference on resource-constrained single-board computer (SBC) clusters, such as Raspberry Pi 5 systems. The proposed architecture dynamically routes tokens to lightweight expert modules, significantly reducing per-token computation while maintaining functional multimodal inference capability. Integrated with Qwen2.5-VL-3B and deployed across a 3-node Raspberry Pi cluster, our approach achieves 48.55 ms inference latency with 81% of time spent in computation rather than communication. Experimental results demonstrate balanced expert utilization, near-linear scalability (1.80× speedup with 3 nodes), and only 17% latency overhead compared to baseline execution (the original Qwen2.5-VL-3B model without MoTE). This work establishes the feasibility of distributed MoE-based VLMs on low-cost edge hardware.

CCS Concepts

• **Computing methodologies** → **Machine learning**; *Distributed computing methodologies*; • **Hardware** → *Embedded systems.*

Keywords

Vision–Language Models, Mixture of Experts, Edge AI, Single-Board Computers, Distributed Inference

ACM Reference Format:
Muhammad Ashir, Hirokazu Madokoro, and Stephanie Nix. 2026. Mixture of Tiny Experts for Resource-Constrained Vision-Language Models on Single Board Computer Clusters. In *Proceedings of 12th International Conference on Computing and Artificial Intelligence (ICCAI 2026).* ACM, New York, NY, USA, 5 pages.

1 Introduction

Vision Language Models (VLMs) [3] merge visual perception and natural language understanding, enabling tasks such as image captioning, visual question answering, and multimodal reasoning. Recent open-source models, including Qwen-VL [1], achieved competitive performance by scaling parameter counts and model depth. Qwen2.5-VL provides strong multimodal reasoning capability and supports model modification through its open architecture. Although smaller variants such as Qwen2-VL [17] 2B exist, the 3B configuration provides sufficient representational capacity for multimodal reasoning while remaining structurally suitable for expert modularization and distributed execution across SBC clusters. However, such dense architectures are primarily optimized for GPU-rich environments and remain impractical for edge deployment. Single-board computers (SBCs) [6], such as Raspberry Pi clusters, offer an efficient alternative due to their low cost, energy efficiency, and deployability in real-world environments. However, their limited memory and compute capabilities make direct deployment of large VLMs infeasible. Even aggressive quantization techniques often fail to address the inefficiency of dense per-token execution.

To address these limitations, this study proposes a Mixture-of-Experts (MoE) [2] based Mixture of Tiny Experts framework for VLMs. The proposed approach decomposes dense transformer [12] blocks into multiple lightweight expert modules and dynamically activates only a subset of experts per token, enabling sparse computation. The primary objective of this research is to demonstrate that expert modularization enables efficient multimodal inference and distributed execution across SBC clusters.

The design is motivated by two key principles: (1) Experts must be small enough to execute on SBC hardware and (2) The architecture should support scalable distributed execution across multiple SBC nodes. This work builds upon prior investigation of Qwen-VL deployment on cluster environments and extends it toward expert-based modularization [4]. Recent advances in distributed LLM inference on SBCs [16] and comprehensive surveys of MoE deployment [14] further motivate our approach, which represents the first application of these techniques to vision-language models.

2 Related Work

2.1 Vision Language Models

Modern VLMs combine a visual encoder with a large language model backbone. Qwen2.5-VL integrates instruction-following language models with visual token embeddings, enabling flexible multimodal interaction. While effective, its dense transformer execution incurs high memory and compute costs.

2.2 Edge AI and Model Compression

Edge AI research has explored model compression techniques, including pruning and quantization [9], as well as 8-bit integer quantization for transformer inference [5], to reduce deployment cost. Recent surveys [15] have examined VLM deployment on edge networks, highlighting the gap between model capability and hardware constraints.

ICCAI 2026, Okinawa, Japan
2026.

2.3 Comparison with Existing Approaches

Table 1 summarizes the key differences between our work and existing MoE-based approaches.

Table 1: Comparison with Existing MoE-Based Approaches

Work	Hardware	Key Feature
Mobile V-MoEs [4]	GPU	Mobile-friendly ViT
Speculative MoE [8]	GPU	Parallel inference
Ours	**Raspberry Pi 5**	**Distributed SBC**

To our knowledge, this is the first implementation of a Mixture-of-Experts vision-language model deployed across a distributed cluster of single-board computers.

2.4 Mixture-of-Experts Architectures

Mixture-of-Experts (MoE) architectures [4] reduce computation by activating only a subset of model parameters for each token. For an input token x, let:

- $g(x) \in \mathbb{R}^N$ denote the gating network output vector, where N is the number of experts.
- $g_i(x)$ be the i-th component of $g(x)$, representing the normalized gating weight (probability) assigned to expert i, with $\sum_{i=1}^{N} g_i(x) = 1$.
- $\mathcal{S}(x)$ denote the set of selected expert indices for input token x, typically the top-k experts with the highest gating weights.
- $E_i(x)$ be the output of the i-th expert network processing token x.
- y be the final output of the MoE layer after combining selected expert outputs.

The output of the MoE layer is computed as a weighted combination of the selected experts:

$$y = \sum_{i \in \mathcal{S}(x)} g_i(x)\, E_i(x), \tag{1}$$

Most existing MoE models assume large expert networks and high-bandwidth interconnects, which makes them unsuitable for deployment on resource-constrained devices such as SBCs. In contrast, our work focuses on designing tiny experts tailored for low-resource hardware environments.

3 Proposed Architecture

3.1 System Overview

The proposed MoTE-VLM architecture integrates vision encoding with a MoTE-enabled transformer backbone. The system consists of two main components: a vision encoder that extracts visual embeddings from input images, and a text embedding layer that processes input tokens. These embeddings are directly fed into the MoTE framework. The vision encoder is designed to remain modular and replaceable. Although earlier edge deployments relied on lightweight convolutional architectures such as MobileNet [7], recent transformer-based encoders offer improved multimodal representation capability.

As shown in Figure 1, the overall architecture consists of two distinct parts: preprocessing components and the MoTE framework.

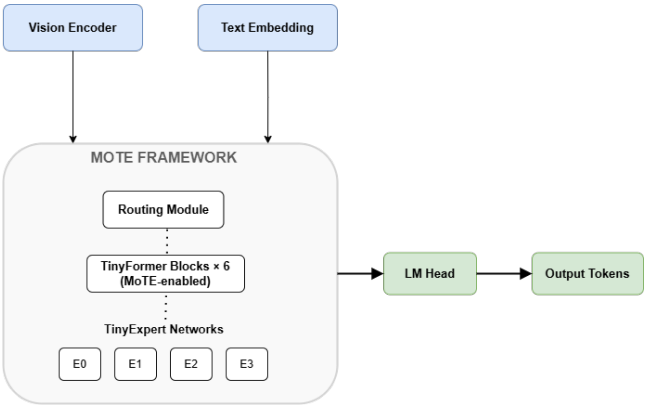


Figure 1: MoTE-VLM architecture overview. The MoTE framework (routing module, TinyFormer blocks, and TinyExpert networks) processes vision and text embeddings to generate output tokens via the LM head.

The MoTE framework comprises three key elements: (i) a token-level routing module that dynamically selects experts, (ii) a set of lightweight TinyExpert networks that process assigned tokens, and (iii) MoTE-enabled TinyFormer blocks that integrate routing and expert computation. This separation clarifies that MoTE replaces the dense feed-forward networks in standard transformers with sparse expert activation.

Let I denote an input image and $T = (t_1, \ldots, t_n)$ denote a sequence of input text tokens. The vision encoder produces visual tokens $V = f_v(I)$, while the text embedding layer generates token embeddings $X = f_t(T)$. These representations are passed directly to the MoTE framework for processing.

The combined representations are processed by a stack of MoTE-enabled transformer blocks, enabling efficient multimodal reasoning through sparse expert [4] activation under resource-constrained settings. Communication between nodes is performed over Gigabit Ethernet using lightweight socket-based transmission. The distributed architecture enables scalability by allowing additional nodes to host new expert modules without modifying the core model structure.

3.2 Mixture of Tiny Experts Design

Each expert module is implemented using a lightweight transformer-inspired feed-forward network derived from the TinyFormer [10] design philosophy. The expert architecture prioritizes parameter efficiency by reducing hidden dimensionality and removing computationally expensive attention layers while preserving nonlinear transformation capability.

Let $z \in \mathbb{R}^d$ denote the input token embedding, where d is the model dimension. Each expert performs two-stage projection with nonlinear activation:

$$E(z) = W_2\, \sigma(W_1 z), \tag{2}$$

where $W_1 \in \mathbb{R}^{h \times d}$ and $W_2 \in \mathbb{R}^{d \times h}$ are learnable weight matrices, h denotes the reduced hidden dimension, and $\sigma(\cdot)$ represents a nonlinear activation function.

3.3 Routing Mechanism

$$r = \text{Softmax}(W_r z + b_r), \tag{3}$$

where $W_r \in \mathbb{R}^{N \times d}$ and $b_r \in \mathbb{R}^N$ are learnable parameters. Here, $r \in \mathbb{R}^N$ is a probability distribution over experts, with $\sum_{i=1}^{N} r_i = 1$. The set of selected experts is defined as:

$$\mathcal{S}(z) = \text{TopK}(r, k), \tag{4}$$

where $\text{TopK}(r, k)$ returns the indices of the k largest values in r. In our implementation, we use $k = 1$ (top-1 routing), meaning each token is sent to exactly one expert, prioritizing latency efficiency over representational capacity.

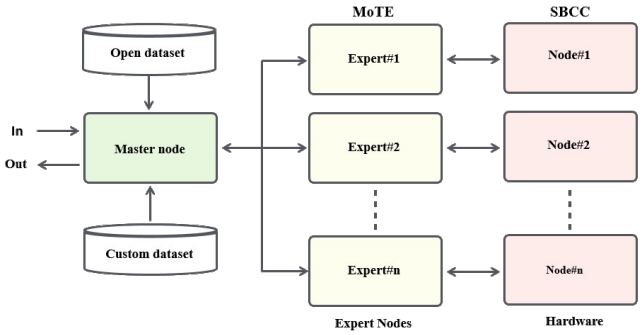


Figure 2: Token routing and expert selection mechanism. Input tokens are processed by the master node's router, which assigns each token to expert nodes based on gating scores. Each expert node processes its assigned tokens in parallel and returns the results to the master node for reassembly.

3.4 Training Objective

To prevent routing collapse and ensure balanced utilization across experts, an auxiliary load balancing loss is introduced [13]. Let count_i denote the number of tokens assigned to expert i, and let $T = \sum_{i=1}^{N} \text{count}_i$ be the total number of tokens in the batch. The load balancing objective is:

$$L_{\text{balance}} = \frac{1}{N} \sum_{i=1}^{N} \left(\frac{\text{count}_i}{T} - \frac{1}{N} \right)^2. \tag{5}$$

The final training objective combines language modeling and load balancing terms:

$$L = L_{LM} + \lambda L_{\text{balance}}, \tag{6}$$

where L_{LM} denotes the standard autoregressive language modeling loss and λ controls the trade-off between the two objectives. In our experiments, we set $\lambda = 0.1$.

4 Experimental Setup and Preliminary Results

4.1 Experimental Setup

Experiments were conducted using a cluster of Raspberry Pi 5 SBCs interconnected through a Gigabit Ethernet switch. Each SBC node hosts a TinyExpert module that receives token embeddings from a centralized controller and returns processed embeddings through a TCP-based communication interface. All expert inference experiments on SBC nodes were performed using CPU-only execution. The main VLM execution was performed on a workstation GPU to simulate hybrid edge-cloud deployment. Table 2 summarizes the hardware configuration.

Table 2: SBC Cluster Hardware Specifications

Component	Specification
Device Model	Raspberry Pi 5
CPU	Quad-core ARM Cortex-A76 @ 2.4 GHz
Memory	8 GB LPDDR4X
Storage	microSD / SSD
Network	Gigabit Ethernet
Operating System	Raspberry Pi OS (64-bit)
Number of Nodes	3
Acceleration	CPU-only Execution

The MoTE framework was integrated with the Qwen2.5-VL-3B-Instruct model. The processing pipeline consists of three stages: (1) multimodal embedding generation using the base VLM, (2) distributed token routing through the MoTE controller, and (3) output generation from MoTE-processed embeddings.

4.2 Training Stability Validation

The preliminary training experiment was conducted on images from the COCO [18] validation dataset paired with generic text prompts, processed using the Qwen2.5-VL tokenizer with a vocabulary of 151,643 tokens. The model was optimized with a learning rate of 3×10^{-4} for 350 epochs (17,500 steps) with a batch size of 4. The loss drops sharply from 11.2 at initialization to below 0.20 within the first 200 steps, then stabilizes around 0.16 for the remaining training, confirming stable convergence under sparse top-1 routing without routing collapse.

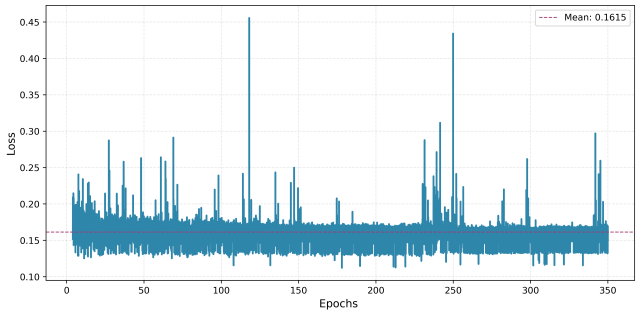


Figure 3: Training loss of MoTE-VLM over 350 epochs on 200 COCO 2017 images, shown on a logarithmic scale. The loss drops sharply from 11.2 at initialization to below 0.20 within the first few epochs, then stabilizes around 0.16 throughout the remaining training, confirming stable convergence under sparse top-1 routing without routing collapse.

The model was trained under the combined objective defined in Eq. 6. Although minor fluctuations are observed, the loss exhibits overall stable behavior without divergence or routing collapse. This

confirms that: (1) gradient-based optimization remains stable under sparse top-1 routing, (2) the load balancing loss (Eq. 5) does not destabilize language modeling optimization, and (3) the routing mechanism distributes tokens without collapsing to a single expert.

Having confirmed routing stability through preliminary training, we next evaluate the distributed inference performance of the MoTE-VLM pipeline on the SBC cluster.

4.3 Distributed Inference Performance

The controller achieved an overall routing latency of 48.55 ms while processing 214 multimodal tokens, confirming stable distributed inference across SBC devices. Table 3 presents the measured distributed inference latency.

Table 3: Distributed SBC Expert Latency During MoTE Execution

Expert	Connect (ms)	Transfer (ms)	Total (ms)
Expert 0	0.29	43.57	43.86
Expert 1	1.56	41.95	43.51
Expert 2	0.97	34.75	35.72

The lower latency for Expert 2 (35.72 ms) reflects fewer tokens routed to this expert (42 vs. 86 for Experts 0 and 1), demonstrating effective load distribution. The integrated MoTE-VLM pipeline successfully generated coherent multimodal descriptions from real image-text inputs, demonstrating that routing embeddings through TinyExpert modules does not degrade multimodal reasoning capability.

4.4 Efficiency and Scalability Analysis

The power consumption of each Raspberry Pi 5 was estimated based on the vendor's typical power specifications for CPU-intensive workloads, measured under sustained expert inference. Actual power draw may vary depending on CPU frequency, network activity, and thermal conditions.

Table 4: Estimated Power Consumption Comparison. GPU power is reported using vendor-specified TDP, while SBC power values correspond to estimated actual power consumption under sustained workloads.

System Configuration	Power
Centralized GPU (NVIDIA RTX 4090)	180 W (TDP)
Single SBC Expert Node	7 W
Three SBC Expert Nodes	21 W

Table 5 presents latency measurements across cluster sizes.

The results demonstrate near-linear latency reduction as additional expert nodes are introduced, with total latency decreasing from 87.24 ms for 1 node to 48.55 ms for 3 nodes. The remaining 5.20 ms accounts for master-node routing logic and token reassembly between expert returns and the next transformer layer. Table 6 provides a detailed breakdown of the latency components.

Table 5: Scalability Analysis with Increasing SBC Nodes

Number of SBC Experts	Total Latency (ms)
1 Node	87.24
2 Nodes	63.10
3 Nodes	48.55

Table 6: Communication and Computation Latency Breakdown

Component	Mean Latency (ms)
TCP Connection Setup	0.94
Token Transmission	4.28
Expert Computation	35.02
Response Transmission	3.11
Routing & Reassembly	5.20
Total	**48.55**

4.5 Evaluation of Execution Modes

Table 7 presents the comparative latency results between baseline and MoTE-enhanced inference. Sequential MoTE execution introduces additional routing and communication overhead (71.85 ms), while distributed parallel execution significantly reduces latency to 48.55 ms, demonstrating the effectiveness of expert parallelization. Although MoTE introduces slight latency overhead compared to baseline execution, it enables scalable distributed inference.

Table 7: Comparison of Baseline VLM and MoTE-Enhanced Inference

Configuration	Inference Latency (ms)
Baseline VLM (No MoTE)	41.32
MoTE Sequential Experts	71.85
MoTE Parallel SBC Experts	48.55

5 Discussion

5.1 Key Findings

Our experiments yield three main insights for distributed MoE on edge devices.

Computation Dominates Communication. Expert computation accounts for 81% of total latency (35.02 ms), while communication overhead remains below 20% (Table 6). This computation-dominant behavior reflects three factors: (i) each TinyExpert contains approximately 2.1M parameters ($2048 \rightarrow 512 \rightarrow 2048$), requiring substantial matrix multiplications; (ii) the Raspberry Pi 5's ARM Cortex-A76 CPU handles all computation without GPU acceleration; and (iii) our use of a dedicated Gigabit Ethernet switch and TCP connection reuse keeps network overhead low.

Load Balancing is Effective. The load balancing loss (Eq. 5) prevents routing collapse. Expert utilization varies by less than 15% from uniform distribution, with expert 2 processing 42 tokens

compared to 86 for experts 0 and 1, demonstrating effective load distribution.

Scalability is Near-Linear. Adding nodes reduces latency: 1 node (87.24 ms) → 2 nodes (63.10 ms) → 3 nodes (48.55 ms), demonstrating effective parallelization.

5.2 Limitations & Ongoing Work

Current limitations include: (1) training on workstation rather than distributed across SBCs, (2) full benchmark evaluation (VQA-v2) [19] reserved for future work, and (3) validation limited to a 3-node cluster. Ongoing work addresses these through VQA-v2 and COCO captioning evaluation, distributed training implementation, INT8 quantization, and analysis of expert specialization patterns. Additionally, we plan to examine weak scaling [20] in future studies to evaluate system performance when both problem size and computational resources increase proportionally.

6 Conclusion

We presented a Mixture of Tiny Experts (MoTE) framework that enables vision-language models to operate efficiently on single-board computer clusters. By decomposing dense transformer blocks into lightweight, sparsely activated experts, the proposed architecture dramatically reduces per-token computation while preserving multimodal reasoning capability. Integrated with Qwen2.5-VL and deployed across a Raspberry Pi 5 cluster, our approach demonstrates that distributed expert execution is both feasible and effective on low-cost edge hardware.

This work yields three key insights for edge-based multimodal AI: First, expert computation dominates inference latency, validating the TinyExpert design philosophy. Second, load balancing regularization successfully prevents routing collapse. Third, the architecture scales near-linearly with additional nodes, confirming its suitability for cluster-based edge environments.

More broadly, this research establishes that combining sparse Mixture-of-Experts architectures with distributed edge computing is a promising direction for enabling scalable, energy-efficient multimodal intelligence in resource-constrained settings.

References

[1] S. Bai et al. 2025. Qwen2.5-VL Technical Report. *arXiv preprint arXiv:2502.13923*.

[2] Y. J. Kim. 2023. Mixture of Quantized Experts (MoQE): Complementary Effect of Low-bit Quantization and Robustness. *arXiv preprint arXiv:2310.02410*.

[3] Z. Lu. 2025. Small Language Models: Survey, Measurements, and Insights. *arXiv preprint arXiv:2409.15790*.

[4] E. Daxberger. 2023. Mobile V-MoEs: Scaling Down Vision Transformers via Sparse MoE. *arXiv preprint arXiv:2309.04354*.

[5] T. Dettmers. 2022. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *arXiv preprint arXiv:2208.07339*.

[6] G. Qu and K. Huang. 2025. Mobile Edge Intelligence for Large Language Models: A Contemporary Survey. *IEEE Communications Surveys & Tutorials*. https://doi.org/10.1109/COMST.2025.3527641

[7] S. Boddu. 2025. Efficient Edge Deployment of Quantized YOLOv4-Tiny for Aerial Emergency Object Detection on Raspberry Pi 5. *arXiv preprint arXiv:2506.09300*.

[8] Y. Li. 2025. Speculative MoE: Communication Efficient Parallel MoE Inference. *arXiv preprint arXiv:2503.04398*.

[9] J. Lin. 2024. Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. *GetMobile: Mobile Computing and Communications*, 28, 4, 12–17. https://doi.org/10.1145/3714983.3714987

[10] J. Yang, J. Liao, F. Lei, M. Liu, L. Long, J. Chen, H. Wan, B. Yu, and W. Zhao. 2025. TinyFormer: Efficient Sparse Transformer Design and Deployment on Tiny Devices. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*. https://doi.org/10.1109/TCAD.2025.11275632

[11] A. Muhammad, H. Madokoro, and S. Nix. 2025. Optimization and Implementation of Qwen-VL on a Single-Board Cluster Computer. In *Proc. 27th SICE System Integration*, Hiroshima, Japan, 203–204.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 5998–6008.

[13] W. Fedus, B. Zoph, and N. Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research*, 23, 120, 1–39.

[14] Z. Wang, Z. Zhu, T. Xie, and Y. Wang. 2025. A Survey on System and Architecture Optimization Techniques for Mixture-of-Experts Large Language Models. *Journal of Electronics & Information Technology*, 47, 11, 4055–4078. https://doi.org/10.11999/JEIT250407

[15] A. Sharshar et al. 2025. Vision-Language Models for Edge Networks: A Comprehensive Survey. *arXiv preprint arXiv:2502.07855*.

[16] B. Tadych. 2025. distributed-llama: Running Large Language Models on Raspberry Pi Clusters. GitHub Repository. https://github.com/b4rtaz/distributed-llama [Accessed: Mar. 2026].

[17] P. Wang, S. Bai et al. 2024. Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution. *arXiv preprint arXiv:2409.12191*.

[18] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. 2014. Microsoft COCO: Common Objects in Context. In *https://arxiv.org/pdf/1405.0312*

[19] Y. Goyal, T. Khot, D. Summers-Stay, D. Batra, and D. Parikh. 2017. Making the v in VQA Matter: Elevating the Role of Image Understanding in Visual Question Answering. In *https://arxiv.org/pdf/1612.00837*.

[20] HPC Wiki. 2024. Scaling. https://hpc-wiki.info/hpc/Scaling [Accessed: Mar. 2026].